ChatGPT

Fiverr-like MERN Marketplace – SRS & Implementation Guide

Executive Summary: A production-grade freelance marketplace connects *buyers* and *sellers* of services with robust features for security, payments, and real-time communication. This SRS covers all aspects of the MERN (MongoDB, Express, React, Node.js) stack implementation: user roles, system architecture, data models, APIs, authentication, third-party integrations (Stripe for payments, Cloudinary for media, Socket.IO for chat, Redis for caching), and DevOps (Docker, CI/CD). Key goals include secure JWT-based auth, OAuth login, role-based access, real-time chat, an escrow-like payment flow with Stripe Connect, image upload via Cloudinary, analytics, notifications, and admin moderation. We outline a complete folder structure, database schemas, API design, and deployment strategy. Official docs for Stripe, Cloudinary, Socket.IO, Redis, GitHub Actions, etc., are referenced to ensure best practices【5†L18-L24】【18†L2723-L2731】【24†L149-L157】【31†L313-L321】. Mermaid diagrams illustrate the ER model and sequence flows (auth, chat, payment). A multi-stage Docker setup, GitHub Actions CI/CD pipeline, and AWS deployment are recommended. Detailed tables summarize endpoints, environment variables, and project milestones.

1. Project Overview and Goals

A Fiverr-like platform enables **buyers** to browse and purchase service *gigs* (design, coding, etc.) from **sellers**, with an **admin** overseeing quality and compliance. The system must ensure secure transactions, handle payments, and support real-time interactions. Key goals include:

- **User Management & Roles:** Users register as *buyer*, *seller*, or *admin*. Authentication uses JWT and Google OAuth. Role-based access control enforces permissions (e.g. only sellers can create gigs, admins moderate content).
- **Gig Marketplace:** Sellers create/edit service listings with titles, descriptions, pricing, and media. Buyers search, filter, and order gigs.
- **Secure Payments:** Use **Stripe Connect** for marketplace payments. Implement "escrow" via delayed payouts: hold funds until work is delivered (via Stripe's manual payout or automatic transfers)【5†L18-L24】【5†L24-L29】.
- **Real-Time Chat:** Integrate **Socket.IO** for instant messaging between buyers and sellers. Support presence indicators, typing, and read receipts.
- **File Uploads:** Use **Cloudinary** for image upload and CDN delivery. The backend handles image uploads (via Cloudinary's Node SDK) and stores URLs【18†L2723-L2731】.
- **Data & Caching:** MongoDB (via Mongoose) stores users, gigs, orders, etc. **Redis** provides session storage, caching, and rate-limiting; use TTL and LRU eviction policies for efficiency【33†L146-L154】【31†L313-L321】.
- **Analytics & Dashboard:** Sellers see metrics (total sales, earnings, reviews). Compute via MongoDB aggregations (e.g. `$group` with `$sum` for totals)【46†L25-L34】.
- **Admin Panel:** Admins can manage users, gigs, reviews, and view logs of actions.
- **Notifications:** Transactional emails (via SMTP or a service) notify users of orders, messages, and system updates.
- **User Experience:** A modern responsive React UI with dark-mode support using Tailwind CSS (`dark:` variants and toggling)【48†L307-L314】【48†L329-L332】.

- **DevOps:** Use Docker Compose for containerization (Node app, React app, MongoDB, Redis), a robust CI/CD pipeline with GitHub Actions (lint, test, build, Docker push, deploy), and AWS (ECS/ECR or alternatives like Vercel/Render) for production.

2. User Roles and User Stories

Roles:

- **Guest:** Browse featured gigs.
- **Buyer:** Registered user who can search gigs, place orders, chat with sellers, leave reviews, manage a wishlist.
- **Seller:** Can create gigs, manage offers, communicate with buyers, manage orders, view earnings.
- **Admin:** Oversees platform, with access to all users/gigs; can ban users, remove content, and access audit logs.

User Stories: Examples include:

- *As a Seller,* I want to create a new gig with images and pricing, so that buyers can order my service.
- *As a Buyer,* I want to filter gigs by category, price, and rating, so I find relevant services quickly.
- *As a Buyer,* I want to see the seller's past reviews before ordering, to make an informed choice.
- *As a Buyer,* I want to pay securely via Stripe and have funds held until delivery (escrow).
- *As a Buyer or Seller,* I want real-time chat to discuss project details.
- *As a Seller,* I want a dashboard showing total sales and earnings trends.
- *As an Admin,* I want to review flagged gigs or reviews and ban malicious users.
- *As any User,* I want email notifications for order updates and messages.

3. Architecture & Folder Structure

The project follows a clear **client/server separation**, plus services for real-time and storage:

```
/ (root)
├── client/                 # React app (frontend)
│   ├── public/            # Public assets (index.html, manifest)
│   ├── src/
│   │   ├── components/    # React components
│   │   ├── pages/         # Page-level components (Home, Profile, etc.)
│   │   ├── hooks/         # Custom React hooks
│   │   ├── context/       # Context providers (Auth, Theme)
│   │   ├── utils/         # Utils (API calls, constants)
│   │   ├── App.jsx        # Main App component and routing
│   │   └── index.jsx      # Entry point
│   ├── tailwind.config.js  # Tailwind CSS config (dark mode setup)
│   └── package.json
├── server/                 # Node/Express backend
│   ├── src/
│   │   ├── controllers/   # Route handlers (auth, gigs, orders, etc.)
│   │   ├── models/        # Mongoose schemas (User, Gig, Order, etc.)
│   │   ├── routes/        # Express routers
```

```
│   │   ├── middleware/     # Auth middleware (JWT, roles, error handling)
│   │   ├── services/       # Stripe, Cloudinary, email, etc. service code
│   │   ├── sockets/        # Socket.IO handlers (chat, notifications)
│   │   ├── utils/          # Utilities (validators, logger)
│   │   ├── config/         # Config (DB, env vars, rate limiter, CORS)
│   │   └── app.js          # Express app setup
│   ├── worker/             # (Optional) background jobs (e.g. analytics,
email)
│   ├── tests/              # Unit & integration tests
│   └── package.json
├── docker-compose.yml      # Defines MongoDB, Redis, server, client services
├── Dockerfile (server)     # Docker multi-stage build for production
├── Dockerfile (client)     # Dockerfile for building static React app
├── .github/workflows/      # GitHub Actions CI/CD pipelines
└── README.md
```

- **Client:** A React app uses Context for auth state and theme (dark mode). Axios or Fetch calls the backend API. Tailwind CSS is configured for light/dark theming【48†L307-L314】.
- **Server:** Express.js application. Use `express.Router()` modules and middleware for authentication (JWT) and role checks. Sessions not used (stateless JWT), but refresh tokens may be stored in DB (or Redis).
- **Socket.IO:** The server also initializes a Socket.IO server (attached to the HTTP server). Socket handlers in `server/src/sockets/` manage chat rooms and real-time events.
- **Docker:** Two Dockerfiles – one for the Node server, one for the React app – use multi-stage builds to minimize image size【52†L13-L17】. Docker Compose sets up the server, client, MongoDB, and Redis.
- **Environment Config:** Centralized in `server/src/config/index.js`, reading from `process.env` (e.g. `MONGO_URI`, `JWT_SECRET`, `STRIPE_SECRET_KEY`, `CLOUDINARY_*`, etc.). Ensure no secrets are committed; use Docker/CI secrets.

4. Data Models (MongoDB Schemas)

We use **Mongoose** schemas for each entity. Key models include:

```
// User model
const UserSchema = new Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true, lowercase: true },
  password: { type: String, required: true, select: false }, // hashed
  role: { type: String, enum: ['buyer','seller','admin'], default: 'buyer' },
  googleId: { type: String },       // for Google OAuth
  // Optional profile info
  avatar: { type: String },         // URL to Cloudinary image
  bio: { type: String },
  // Additional security tokens (e.g. refresh token hash)
  refreshToken: { type: String, select: false }
}, { timestamps: true });
```

```
UserSchema.index({ email: 1 });
module.exports = mongoose.model('User', UserSchema);
```

```
// Gig model
const GigSchema = new Schema({
  seller: { type: mongoose.Types.ObjectId, ref: 'User', required: true },
  title: { type: String, required: true, index: 'text' },
  description: { type: String, required: true, index: 'text' },
  category: { type: String, required: true, index: true },
  price: { type: Number, required: true },
  deliveryDays: { type: Number, default: 7 },
  images: [{ public_id: String, url: String }],  // Cloudinary IDs/URLs
  // Ratings
  totalReviews: { type: Number, default: 0 },
  avgRating: { type: Number, default: 0 },
  // Status
  isActive: { type: Boolean, default: true }
}, { timestamps: true });
// Text index for search
GigSchema.index({ title: 'text', description: 'text' });
module.exports = mongoose.model('Gig', GigSchema);
```

```
// Order model
const OrderSchema = new Schema({
  gig: { type: mongoose.Types.ObjectId, ref: 'Gig', required: true },
  buyer: { type: mongoose.Types.ObjectId, ref: 'User', required: true },
  seller: { type: mongoose.Types.ObjectId, ref: 'User', required: true },
  price: { type: Number, required: true },
  status: { type: String, enum: ['pending','paid','completed','cancelled'],
default: 'pending' },
  startDate: { type: Date, default: Date.now },
  dueDate: { type: Date },    // when the seller should deliver
  // Stripe-related fields
  stripePaymentIntentId: { type: String },
  stripeTransferId: { type: String },
  stripePayoutId: { type: String }
}, { timestamps: true });
OrderSchema.index({ buyer: 1 });
OrderSchema.index({ seller: 1 });
module.exports = mongoose.model('Order', OrderSchema);
```

```
// Review model
const ReviewSchema = new Schema({
  gig: { type: mongoose.Types.ObjectId, ref: 'Gig', required: true },
  author: { type: mongoose.Types.ObjectId, ref: 'User', required: true }, //
buyer
  rating: { type: Number, min: 1, max: 5, required: true },
  comment: { type: String },
```

```
}, { timestamps: true });
ReviewSchema.index({ gig: 1 }); // query all reviews for a gig
module.exports = mongoose.model('Review', ReviewSchema);
```

```
// Conversation (Chat) model
const ConversationSchema = new Schema({
  participants: [{ type: mongoose.Types.ObjectId, ref: 'User' }], // buyer
and seller
}, { timestamps: true });
ConversationSchema.index({ participants: 1 });
module.exports = mongoose.model('Conversation', ConversationSchema);

const MessageSchema = new Schema({
  conversation: { type: mongoose.Types.ObjectId, ref: 'Conversation',
required: true },
  sender: { type: mongoose.Types.ObjectId, ref: 'User', required: true },
  text: { type: String, required: true },
  readBy: [{ type: mongoose.Types.ObjectId, ref: 'User' }], // for read
receipts
}, { timestamps: true });
module.exports = mongoose.model('Message', MessageSchema);
```

```
// Wishlist model
const WishlistSchema = new Schema({
  user: { type: mongoose.Types.ObjectId, ref: 'User', required: true },
  gig: { type: mongoose.Types.ObjectId, ref: 'Gig', required: true }
}, { timestamps: true });
WishlistSchema.index({ user: 1, gig: 1 }, { unique: true });
module.exports = mongoose.model('Wishlist', WishlistSchema);
```

```
// Analytics (sample)
const AnalyticsSchema = new Schema({
  user: { type: mongoose.Types.ObjectId, ref: 'User', required: true }, //
seller or buyer
  metric: { type: String },    // e.g. "revenue", "orders", "visitors"
  value: { type: Number },
  date: { type: Date, default: Date.now }
}, { timestamps: true });
AnalyticsSchema.index({ user: 1, metric: 1, date: 1 });
module.exports = mongoose.model('Analytics', AnalyticsSchema);
```

```
// Admin Log (for moderation actions)
const AdminLogSchema = new Schema({
  admin: { type: mongoose.Types.ObjectId, ref: 'User', required: true },
  action: { type: String, required: true },  // e.g. "ban user", "delete gig"
  targetType: { type: String, enum: ['User','Gig','Review','Order'] },
  targetId: { type: mongoose.Types.ObjectId },
```

```
  reason: { type: String }
}, { timestamps: true });
module.exports = mongoose.model('AdminLog', AdminLogSchema);
```

```
// Notification model
const NotificationSchema = new Schema({
  user: { type: mongoose.Types.ObjectId, ref: 'User', required: true },
  type: { type: String },           // e.g. "order_placed", "message"
  content: { type: String },        // brief message or summary
  isRead: { type: Boolean, default: false }
}, { timestamps: true });
NotificationSchema.index({ user: 1, isRead: 1 });
module.exports = mongoose.model('Notification', NotificationSchema);
```

```
// Session/Token model (optional; for refresh tokens)
const SessionSchema = new Schema({
  user: { type: mongoose.Types.ObjectId, ref: 'User', required: true },
  refreshToken: { type: String, required: true },
  expiresAt: { type: Date, required: true }
});
SessionSchema.index({ refreshToken: 1 });
module.exports = mongoose.model('Session', SessionSchema);
```

Each model uses appropriate **indexes** for fast querying (e.g. user and gig IDs). Text indexes on `Gig.title/description` enable search. Mongoose's schema indexing is configured in code【39†L1-L8】. For example, adding `GigSchema.index({ title: 'text', description: 'text' })` allows efficient full-text search.

5. REST API Design

The backend exposes a RESTful JSON API with proper error codes. All endpoints under `/api/...`, using Express. Key routes include:

Method	Endpoint	Description	Auth	Roles
POST	`/api/auth/register`	Register new user (buyer or seller)	No	Guest
POST	`/api/auth/login`	Log in with email/password (returns JWT & refresh token)	No	Guest
POST	`/api/auth/google`	Google OAuth login (server handles OAuth flow)	No	Guest
POST	`/api/auth/refresh`	Refresh JWT using refresh token	Yes	All
POST	`/api/auth/logout`	Revoke refresh token	Yes	All
GET	`/api/users/me`	Get own profile	Yes	User

Method	Endpoint	Description	Auth	Roles
PUT	`/api/users/me`	Update own profile	Yes	User
GET	`/api/users/:id`	Get other user (seller) profile	Yes	Admin/ User
DELETE	`/api/users/:id`	Delete user (admin only)	Yes	Admin
GET	`/api/gigs`	List/search gigs (with query params: text, category, price range, rating)	No	Any
GET	`/api/gigs/:id`	Get gig details	No	Any
POST	`/api/gigs`	Create new gig	Yes	Seller
PUT	`/api/gigs/:id`	Update own gig	Yes	Seller
DELETE	`/api/gigs/:id`	Delete own gig	Yes	Seller
GET	`/api/orders`	List orders (filtered by buyer or seller based on role)	Yes	User/ Seller
POST	`/api/orders`	Create new order (place an order & initiate Stripe payment)	Yes	Buyer
GET	`/api/orders/:id`	Get order by ID (for buyer/seller/ admin)	Yes	User/ Admin
PUT	`/api/orders/:id/ complete`	Mark order as completed (post-delivery)	Yes	Seller
POST	`/api/conversations`	Create/get conversation (buyer-seller chat channel)	Yes	User/ Seller
GET	`/api/conversations/:id/ messages`	Get messages in conversation (pagination support)	Yes	User/ Seller
POST	`/api/messages`	Send a message (via REST fallback for chat)	Yes	User/ Seller
GET	`/api/reviews/:gigId`	Get all reviews for a gig	No	Any
POST	`/api/reviews/:gigId`	Add review to a gig (only after order complete)	Yes	Buyer
POST	`/api/wishlist`	Add gig to wishlist	Yes	Buyer
GET	`/api/wishlist`	List own wishlist	Yes	Buyer
DELETE	`/api/wishlist/:gigId`	Remove gig from wishlist	Yes	Buyer
POST	`/api/stripe/create-checkout-session`	Create Stripe Checkout Session/ PaymentIntent	Yes	Buyer
POST	`/api/webhooks/stripe`	Stripe webhook endpoint (signature-verified)	No	–

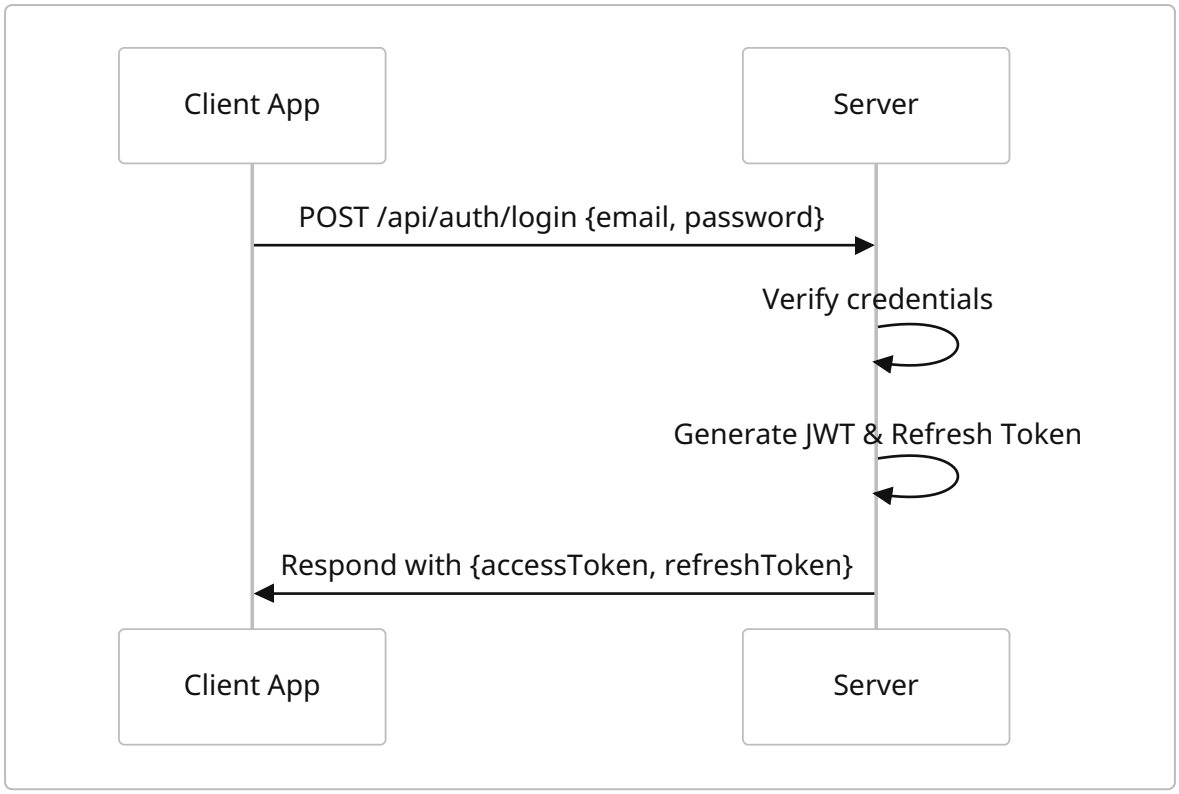
Method	Endpoint	Description	Auth	Roles
POST	`/api/notifications/:id/read`	Mark notification as read	Yes	User
GET	`/api/admin/users`	List all users	Yes	Admin
PUT	`/api/admin/users/:id/ban`	Ban a user	Yes	Admin
GET	`/api/admin/gigs`	List all gigs (for moderation)	Yes	Admin
PUT	`/api/admin/gigs/:id/disable`	Disable a gig	Yes	Admin
GET	`/api/admin/logs`	Get admin action logs (pagination)	Yes	Admin

Authentication middleware (`authMiddleware`) verifies JWTs on protected routes; role-check middleware (e.g. `permit('seller')`) enforces access by user role【42†L216-L222】. Responses use standard HTTP codes (200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 500 Server Error). Errors include messages and codes for frontend handling.

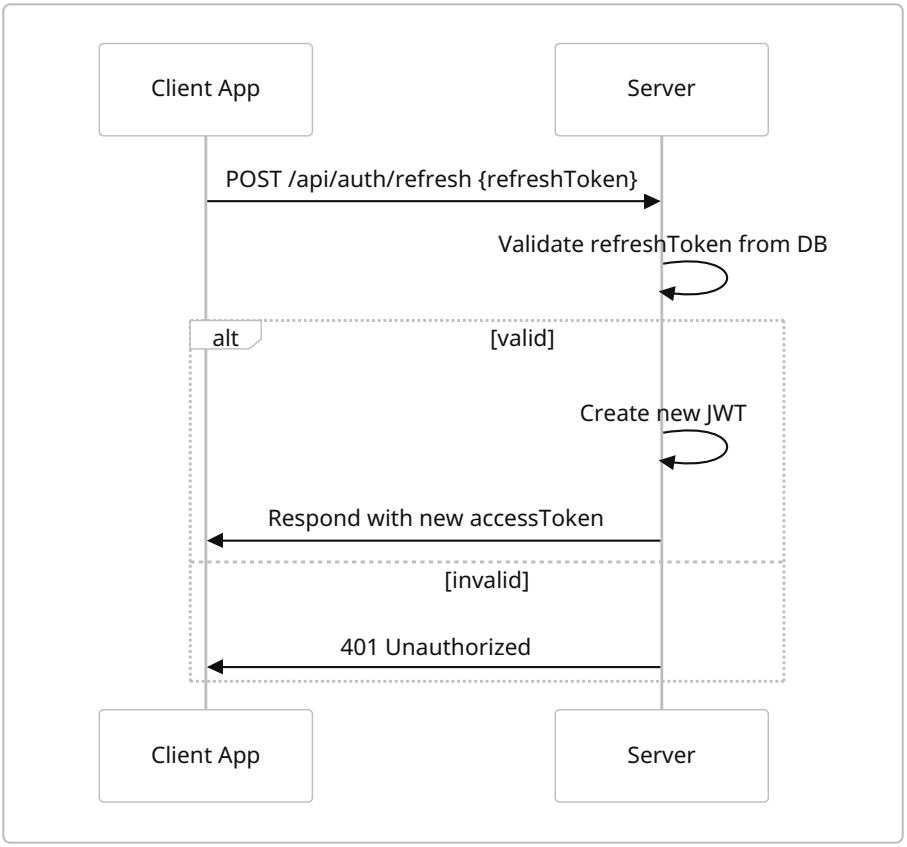
6. Authentication Flows

JWT & Refresh Tokens

Users log in with email/password or Google. On successful login, the server issues a short-lived **access token** (JWT, e.g. 15m) and a long-lived **refresh token** (stored in DB or secure HTTP-only cookie). JWTs embed user ID and role; protected endpoints validate the JWT. Refresh tokens are sent to `/api/auth/refresh` when the client's access token expires, yielding a new JWT. The flow is:



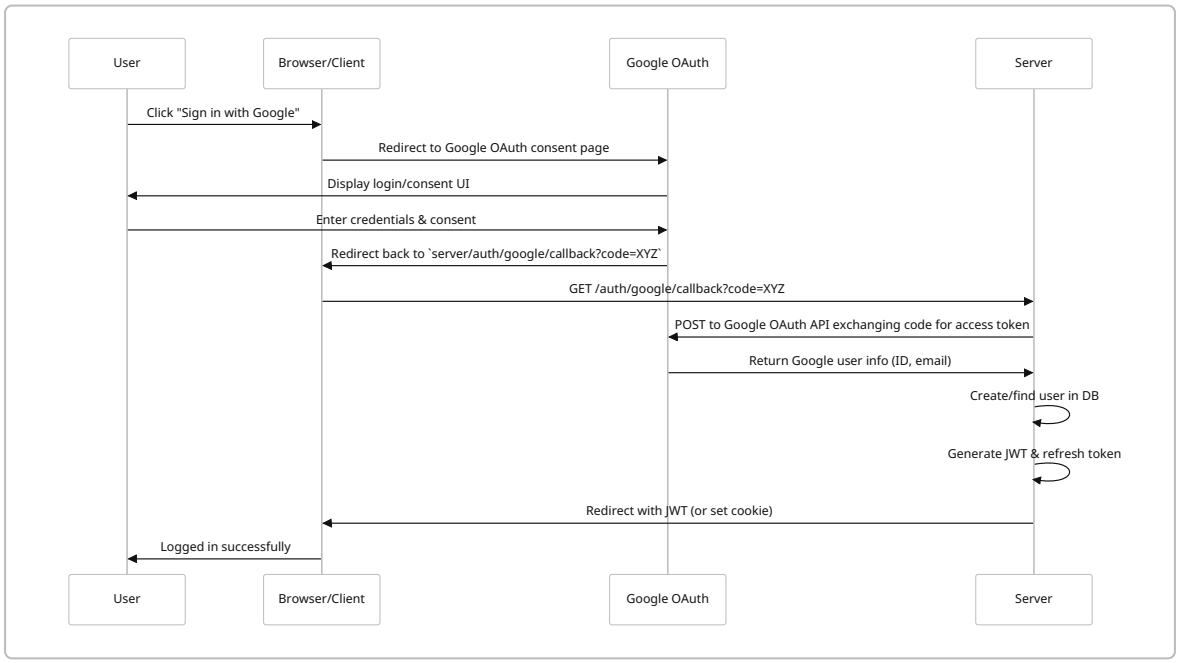
For refresh:



Official guidance encourages using libraries (e.g. `jsonwebtoken`) and best practices (store secret in env vars, rotate keys) to secure JWT handling【35†L164-L168】.

Google OAuth 2.0

For "Sign in with Google", the client redirects the user to Google's OAuth consent page. After consent, Google redirects back with a code, which the server exchanges for tokens and user info【35†L134-L142】. The server then finds or creates the user record and issues a JWT. Sequence:



The Google docs warn **"use OAuth libraries"** for security【35†L164-L168】. We must also handle error cases (denied consent) and link Google accounts by unique `googleId`.

7. Real-Time Chat (Socket.IO)

We implement Socket.IO on Node.js for **bi-directional real-time messaging**【20†L78-L80】. On client connect, authenticate via the JWT (e.g. send token in connection query or cookie). The server verifies the token, identifies the user, and joins them to rooms: one room per conversation or a personal room named by user ID【24†L159-L167】.

- **Rooms:** Each conversation (buyer-seller pair) has a unique room ID (e.g. the Conversation's ObjectId). When a user opens a chat, the client emits `socket.emit('join', conversationId)`, and server `socket.join(conversationId)`.
- **Private Messages:** Socket.IO automatically creates a room for each socket ID【24†L149-L153】. We assign each user's socket(s) to a room named after their user ID: on connect, do `socket.join(userId)`. Then to send a private event, use `io.to(userId).emit(...)` 【24†L159-L167】.
- **Messaging Flow:** When a client sends a message (`socket.emit('message', {conversationId, text})`), the server saves it to MongoDB and broadcasts to the room: `io.to(conversationId).emit('message', newMessage)`. The recipient's client displays it instantly.
- **Read Receipts & Typing:** Clients emit events like `socket.emit('typing', {conversationId})`, server forwards to `socket.broadcast.to(conversationId)` to show "User is typing". On reading a message, emit `socket.emit('read', messageId)`,

server updates the message's `readBy` and emits `io.to(originalSenderId).emit('messageRead', messageId)`.

- **Presence:** Server tracks online users (a map of userId → socket count). On `connect`, set user as online and emit a `userOnline` event to relevant contacts; on `disconnect`, emit `userOffline` if no more sockets remain. This allows showing online indicators.

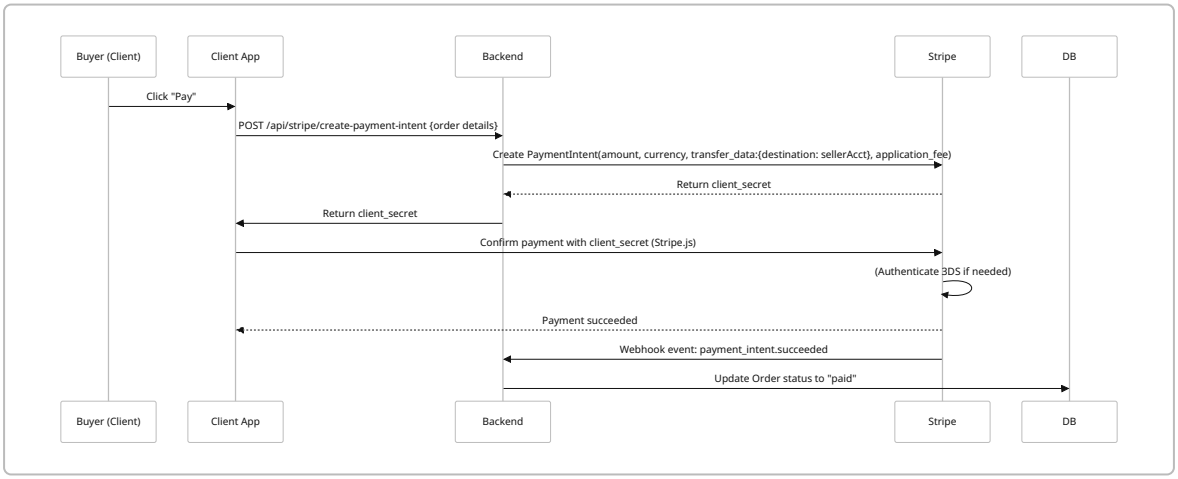
Socket.IO's room system and broadcast make chat easy【24†L149-L153】【24†L159-L167】. For scaling (multiple server instances), a Redis adapter can synchronize events across nodes.

8. Payments with Stripe

We integrate **Stripe Connect** to handle marketplace payments. The platform uses *destination charges* or separate charges/transfers to move funds between buyer, platform (application fee), and seller【5†L18-L24】. The flow is:

- **Onboarding Sellers:** Each seller must connect a Stripe account (standard or express). Use Stripe Connect OAuth (or Stripe Dashboard links) to obtain a `stripeAccountId` for the seller.

- **Checkout:** When a buyer orders a gig, the backend creates a Stripe **PaymentIntent** or **Checkout Session** specifying `transfer_data[destination] = seller.stripeAccountId` and an `application_fee_amount` (the platform fee). This charges the buyer's card and holds funds on the seller's balance (Stripe Connect Docs).



- **Webhooks & Security:** The server must expose a `POST /api/webhooks/stripe` endpoint to handle events (e.g. `payment_intent.succeeded`, `charge.refunded`). Verify Stripe's signature header to ensure authenticity【8†L70-L74】【7†L69-L72】. Webhook handlers update order records, create transfers/payouts to sellers, and send email notifications. For example, after delivery, the platform may call `stripe.transfers.create({ amount: <net amount>, currency, destination: sellerAcct })`. Because Stripe doesn't offer native escrow, the platform can **delay payouts**: the admin or an automated job completes payout at milestone (e.g. after buyer accepts work). Stripe allows scheduling payouts up to 90 days later (varies by country)【5†L24-L29】.

- **Refunds & Security:** If an order is refunded, the webhook updates statuses and performs Stripe refunds. **Always use HTTPS** and verify event signatures to prevent spoofing【8†L70-L74】. Do not expose secret keys: use Stripe's official Node SDK on the backend.

- **Compliance:** Stripe handles PCI compliance for card data if using Stripe.js/tokenization. Never log or store raw card info. Follow Stripe's security guide to use TLS and verify signatures【8†L70-L74】.

9. Cloudinary Image Management

All user-uploaded images (gig images, avatars, etc.) are stored on **Cloudinary**, a cloud CDN for media. The backend uses the official Cloudinary Node SDK【11†L36-L40】. The flow is: the client uploads files to the server (e.g. via a React `<input>` posting `FormData`), or directly to Cloudinary with a signed upload preset. The server then calls:

```
const result = await cloudinary.v2.uploader.upload(filePath, { folder:
'gigs' });
```

Cloudinary's API accepts local file paths, remote URLs, Buffers or streams, etc.【18†L2723-L2731】. On success, Cloudinary returns a public ID and URL, which we store in MongoDB (e.g. in `Gig.images`). We only save references, not the binary data. Advantages: global CDN delivery, on-the-fly transformations, and optimized caching.

Use **unsigned upload presets** (via Cloudinary dashboard) if allowing direct client-side uploads (limits files to 100 MB). For higher security, do server-side signed uploads. After upload, images can be transformed via URL parameters (auto-resize, format).

There is **no rate limit** issue for image uploads up to account limits【18†L2735-L2740】. We should secure API keys: store `CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_API_KEY`, and `CLOUDINARY_API_SECRET` in env vars. Using the SDK ensures uploads over HTTPS.

Cloudinary provides an NPM package (`cloudinary`) as shown above, making integration straightforward【11†L36-L40】.

10. Caching with Redis

We use **Redis** as an in-memory data store for caching, session/token storage, and rate limiting. Redis offers sub-millisecond operations, making it ideal for these roles【31†L313-L321】. Key strategies:

- **Sessions/Refresh Tokens:** (If needed) Store refresh tokens or session data in Redis with a TTL. This allows quick lookup and revocation (e.g. logout deletes key).
- **Cache Hot Data:** Cache frequent read data, such as popular gig lists or user sessions. For example, cache results of expensive queries (last 24h sales stats) with an expiration (e.g. 5min) to reduce DB load.
- **Rate Limiting:** Implement per-user or per-IP rate limits to prevent abuse. A simple fixed-window limiter: use `INCR` on a key (e.g. `ratelimit:userId:minute`) and set `EXPIRE` to 60s【31†L438-L442】. If the counter exceeds a threshold (e.g. 100), block the request. Redis atomic operations and TTL make this efficient. You can also implement sliding-window or token-bucket algorithms (see Redis tutorials for patterns)【31†L313-L321】【31†L438-L442】.
- **Leaderboard/Analytics Cache:** Cache computed dashboard metrics (total sales, top gigs) so sellers see instant updates without recalculating. You could store daily aggregates in Redis for fast retrieval.

- **Eviction Policy:** Configure Redis `maxmemory-policy` to an LRU (Least Recently Used) or TTL-based eviction to avoid memory overuse【33†L146-L154】. For example, use `volatile-lru` to evict the least-recently used keys that have an expiration. Always set appropriate `EXPIRE` on cache entries.

Enable **TLS/SSL** for Redis connections in production, and require authentication. Do not expose Redis to the public internet. Use strong auth passwords or certificates (Redis Enterprise allows mTLS)【56†L91-L99】.

11. Pagination, Search & Filtering

The API supports paginated results and rich search/filter queries:

- **Pagination:** Use query parameters like `?page=2&limit=20` on list endpoints. In Mongoose, implement `.skip((page-1)*limit).limit(limit)`. Return metadata (`total`, `page`, `pages`) in responses. For large offsets, cursor-pagination can be considered.
- **Filtering:** Allow filters via query string (e.g. category, price range, seller). Build Mongoose `find` queries conditionally. Index frequently filtered fields: we indexed `Gig.category`, `price`, etc. Use ranges with `$gte` / `$lte` for numeric fields.
- **Full-Text Search:** For keyword search on gigs, use MongoDB's text indexes (one per collection). We defined a text index on `Gig.title` and `Gig.description`【18†L2723-L2731】. Queries use `{$text: {$search: query}}` and can include text score sorting. Ensure appropriate weights if needed.
- **Sorting:** Support sort parameters (e.g. `?sort=price_asc`, `sort=-rating`). Use Mongoose `.sort()` on queries. Index sorting fields for performance.

Overall, maintain efficient indexes for search and filter: e.g., `{ category: 1, price: 1, "avgRating": -1 }` as needed. Compound indexes (Mongoose's `schema.index({ ... })`) can optimize combined filters【39†L1-L8】.

12. Seller Dashboard Analytics

Sellers need insight into their performance. Dashboard metrics include: total orders, revenue, average earnings per order, and growth. We can compute via MongoDB aggregations. Example aggregation to get monthly sales:

```
db.orders.aggregate([
  { $match: { seller: ObjectId("...") } },
  { $group: {
      _id: { year: {$year: "$createdAt"}, month: {$month: "$createdAt"} },
      totalOrders: { $sum: 1 },
      totalRevenue: { $sum: "$price" }
    } },
  { $sort: { "_id.year": 1, "_id.month": 1 } }
]);
```

This yields year/month buckets with counts and sums. The $group operator with $sum is standard in MongoDB【46†L25-L34】. Similar pipelines can compute daily or weekly trends. These stats can be

cached in Redis or precomputed nightly (via a cron job updating `Analytics` docs) to speed up the dashboard.

Common metrics:

- **Total Sales & Revenue:** Sum orders and price (as above).
- **Pending & Completed Orders:** Count by status.
- **Earnings Graph:** Time-series (daily/weekly).
- **Top Gigs:** Aggregate orders by gig ID.
- **Ratings Distribution:** Query the Reviews collection.

These queries use MongoDB aggregation (e.g., `{$match: {seller: ...}}, {$group: ...}`) 【46†L25-L34】, and should leverage indexes on `seller` and `createdAt`.

13. Reviews & Ratings

Buyers can leave one **review and star-rating** per completed order. Key points:

- Enforce one review per (buyer, gig) after order completion to prevent spam.
- When a new review is saved, update the corresponding `Gig.avgRating` and `Gig.totalReviews`. For example:

```
gig.avgRating = (gig.avgRating * gig.totalReviews + newRating) /
(gig.totalReviews + 1);
gig.totalReviews += 1;
gig.save();
```

- Display reviews on gig page and include average rating.
- To prevent fraud, ensure the buyer actually purchased (order.status == 'completed') and optionally after a cooldown period (e.g. 7 days).
- Admins can remove fake or inappropriate reviews via the admin panel.

We can also implement review helpful votes, but at minimum track count and average. If a review is deleted, decrement totals accordingly. All calculations happen in backend code or via aggregations on demand.

14. Admin Panel & Moderation

Admins have a special UI (could be part of the React app under an “Admin” section) with the ability to:

- View **all users** and change roles or ban them (set `user.isActive = false`).
- View **all gigs** (active/inactive) and disable content.
- Review flagged reviews or chats.
- Monitor **dashboard stats** (site-wide metrics).
- See **audit logs**: actions stored in `AdminLog`, e.g. “Admin X banned User Y” with timestamps.

Moderation workflow example: Admin clicks “Ban User”; the frontend calls `PUT /api/admin/users/:id/ban` (middleware verifies admin role). The server sets `User.isActive=false` and writes to `AdminLog` (reason = “violation of terms”).

Admin endpoints (like `/api/admin/*`) use middleware to ensure `req.user.role === 'admin'`. All admin actions should be logged in the `AdminLog` model for audit trails.

15. Wishlist / Favorites

Each buyer can **favorite** gigs. We model this with the `Wishlist` collection linking user and gig (as shown above). API endpoints allow:

- Adding: `POST /api/wishlist` with `{ gigId }`.
- Listing: `GET /api/wishlist` returns gig details.
- Removing: `DELETE /api/wishlist/:gigId`.

On the frontend, “heart” buttons call these endpoints. The wishlist provides quick access for buyers to saved gigs.

16. Email Notifications

Transactional emails keep users informed. Integrate a mail service (e.g. SendGrid, SES, or nodemailer with SMTP). Use templates (e.g. Handlebars) for these flows:

- **User Actions:** Welcome email on signup, password reset (with token link).
- **Order Events:** Order confirmation to buyer, new order notification to seller, order completion confirmation.
- **Chat:** If offline, email notifications for new messages.
- **System Alerts:** Admin announcements (if any).

Every email is sent from the server via a service. For example, on order placement, `Mailer.sendOrderConfirmation(buyerEmail, orderDetails)`. Store an EmailLog if needed. Templates include order details, links to the app, and clear instructions. Ensure to configure SPF/DKIM for deliverability. Use environment variables for SMTP credentials.

Example Email Template (Order Confirmation)

```
Subject: Your Order #{{order.id}} has been placed

Hi {{buyer.name}},

Thank you for your purchase of "{{gig.title}}" by {{seller.name}}. We have
received your payment and the seller has been notified.

Order ID: {{order.id}}
Amount: ${{order.price}}

You will be notified when the seller delivers your work. If you have any
questions, reply to this email.

Best regards,
Freelance Marketplace Team
```

17. Frontend Theming & Dark Mode

The React UI supports **light/dark themes**. Using Tailwind CSS, we use the `dark:` utility variant (e.g. `bg-white dark:bg-gray-800`) to style components in dark mode【48†L307-L314】. By default, Tailwind uses the OS preference (`prefers-color-scheme`). For a manual toggle, we override to apply a `class="dark"` on `<html>`【48†L311-L319】.

Workflow: 1. Include Tailwind's dark mode in `tailwind.config.js` (default `media` or `class`).
2. In React, add a theme context or use a `<button>` that toggles a `dark` class on the `document.documentElement` and saves preference to `localStorage`【48†L329-L332】.
3. Example:

```
useEffect(() => {
  const theme = localStorage.getItem('theme') || 'light';
  document.documentElement.classList.toggle('dark', theme === 'dark');
}, []);
// On button click: toggle 'dark' class and update localStorage.
```

1. CSS: e.g. `<div class="bg-white dark:bg-gray-900 text-gray-900 dark:text-gray-100"> ... </div>`.

This approach ensures quick switching and respects system defaults.

18. Containerization & Deployment (Docker)

We containerize the app for consistent deployment. **Dockerfiles** (multi-stage) are used:

- **Server Dockerfile (Node.js):**

```
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build   # if transpilation needed

FROM node:18-alpine AS runner
WORKDIR /app
COPY package*.json ./
RUN npm ci --production
COPY --from=builder /app/dist ./dist
ENV NODE_ENV=production
CMD ["node", "dist/app.js"]
```

This multi-stage build builds the code then only copies production artifacts, reducing image size【52†L13-L17】.

- **Client Dockerfile (React):**
  Build the React app and serve via nginx or a static server:

```
FROM node:18-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/build /usr/share/nginx/html
EXPOSE 80
```

(Or use `serve` for simplicity.)

- **docker-compose.yml:** Includes services:
- `db` (MongoDB, version-limited), with a volume for data.
- `redis`, with persistence for cache.
- `server`, built from the server Dockerfile, with env vars.
- `client`, built from the client Dockerfile or from `node:18-alpine` + build & serve.

Set environment variables via the Compose file or external `.env`. For production, use managed services or Kubernetes; but Compose works for local or simple deployment. Use the `maxmemory-policy` in redis.conf as needed (e.g. `allkeys-lru`)【33†L146-L154】.

19. CI/CD with GitHub Actions

A full CI/CD pipeline ensures quality. **CI (Continuous Integration)** steps:
1. **Checkout code** (`actions/checkout@v3`).
2. **Setup Node** (`actions/setup-node@v4`)【54†L347-L354】 with a matrix for Node 18.x (and maybe 20.x).
3. **Install dependencies** (`npm ci`).
4. **Run lint** (`npm run lint`) to enforce code style.
5. **Run tests** (`npm test`) including unit and integration tests.
6. **Build** (`npm run build`).
7. **Report**: publish coverage or artifacts if needed.

Use `actions/setup-node` as recommended【54†L347-L354】 to ensure consistent Node versions. E.g.:

```
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
```

```
    strategy:
      matrix:
        node-version: [18.x]
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-node@v4
        with: node-version: ${{ matrix.node-version }}
      - run: npm ci
      - run: npm run lint
      - run: npm test
```

CD (Continuous Deployment) on `main` branch:

- **Docker Build:** Build Docker images for server and client.
- **Push to Registry:** Tag images (e.g. `${{ github.sha }}`) and push to DockerHub or AWS ECR.
- **Deploy:** Use actions like `aws-actions/configure-aws-credentials` and `aws-actions/amazon-ecs-deploy-task-definition` to update an ECS service or use `scp/ssh` to update a VM. Alternatively, for frontend hosting, push to Vercel/Netlify via their Git integration, and for backend, use AWS ECR/ECS or AWS Elastic Beanstalk.

A sample deployment snippet (to AWS ECS) could be:

```
deploy:
  needs: test
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v3
    - uses: actions/setup-node@v4
    - run: npm ci
    - run: npm run build
    - name: Build Docker image
      run: |
        docker build -t ${{ secrets.DOCKER_REGISTRY }}/app:$
{{ github.sha }} .
        echo ${{ secrets.DOCKER_PASSWORD }} | docker login $
{{ secrets.DOCKER_REGISTRY }} -u ${{ secrets.DOCKER_USERNAME }} --password-
stdin
        docker push ${{ secrets.DOCKER_REGISTRY }}/app:${{ github.sha }}
    - name: Deploy to AWS ECS
      uses: aws-actions/amazon-ecs-deploy-task-definition@v1
      with:
        aws-access-key-id: ${{ secrets.AWS_KEY }}
        aws-secret-access-key: ${{ secrets.AWS_SECRET }}
        aws-region: us-east-1
        task-definition: ecs-task-def.json
        service: my-ecs-service
        cluster: my-cluster
```

CI/CD Best Practices: Keep secrets (API keys, credentials) in GitHub Secrets. Only build tested code. Tag releases. Monitor Action logs for failures. For alternative deployment, GitHub can push the client to Vercel by merging (Vercel auto-deploys on main).

20. Maintenance: Backup, Monitoring, Security

Database Backups:

- **MongoDB:** If using MongoDB Atlas, enable automated backups. Otherwise, run `mongodump` to S3 nightly and retain snapshots.
- **Redis:** Enable RDB/AOF persistence (e.g. RDB snapshot every hour). Alternatively use managed Redis with backup (e.g. AWS ElastiCache automated backups).
- Keep backups off-site and tested (periodically restore test).

Monitoring & Logging:

- Use a logging framework (e.g. Winston or Pino) for server logs. Log requests (via Morgan) and errors. Ship logs to a centralized system (AWS CloudWatch, Loggly, etc.).
- Monitor key metrics: CPU, memory, response times. Use APM tools (New Relic, DataDog) or Prometheus/Grafana.
- Set up health checks (e.g. `/health` endpoint returning DB connectivity). Kubernetes or container platform can restart on failure.
- For chat, monitor socket connections and message rates.

Security Best Practices:

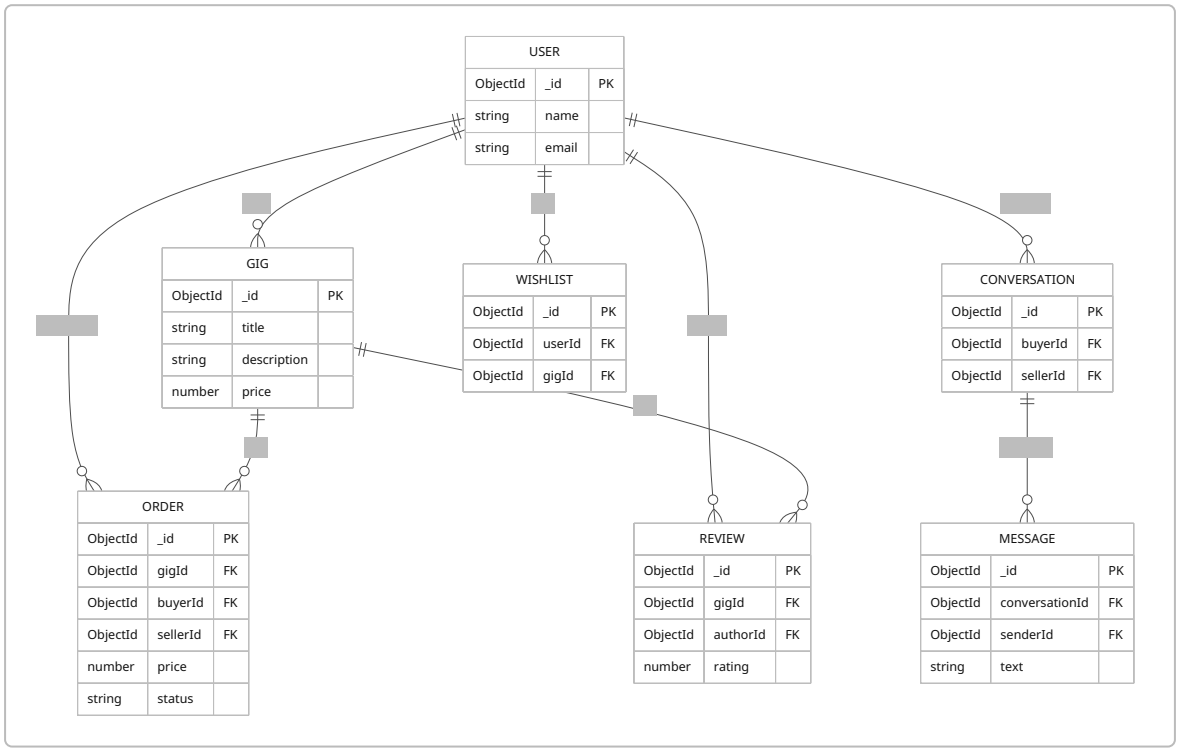
- **Secrets Management:** Never commit keys/credentials. Use env vars, .env files in `.gitignore`, and CI/CD secrets.
- **Transport Security:** Use HTTPS everywhere (certificates via AWS ELB/CloudFront or Let's Encrypt). Stripe/Cloudinary endpoints are HTTPS by default. Use TLS for DB and Redis connections.
- **Authentication:** Store passwords hashed (bcrypt). Use JWT with strong `JWT_SECRET` (rotate periodically). Validate tokens on each request.
- **Authorization:** Enforce role-based access strictly. Validate user permissions on every update/delete.
- **Input Validation:** Sanitize all inputs (e.g. libraries like validator.js) to prevent injection. Use Mongoose schema validation.
- **CORS:** Restrict origins in CORS config to your frontend domain.
- **Rate Limiting & Throttling:** Use Redis-based rate limiter to protect login and API endpoints from brute-force.
- **Data Security:** Do not store sensitive PII in plain text. Encrypt sensitive fields if needed.
- **Dependency Updates:** Keep Node packages up to date, monitor CVEs (e.g. Dependabot).
- **DDoS Protection:** If on AWS, use AWS Shield/CloudFront. On other hosts, limit max connections.
- **Redis Security:** As noted, enable AUTH and TLS【56†L91-L99】, run in a private subnet.
- **Backups & Rollback:** In case of incident, roll back to last stable Docker image, and restore DB from backup if corrupted.

In sum, regular backups, comprehensive logging, and strict security controls (HTTPS, validation, auth) are paramount.

21. System Diagrams

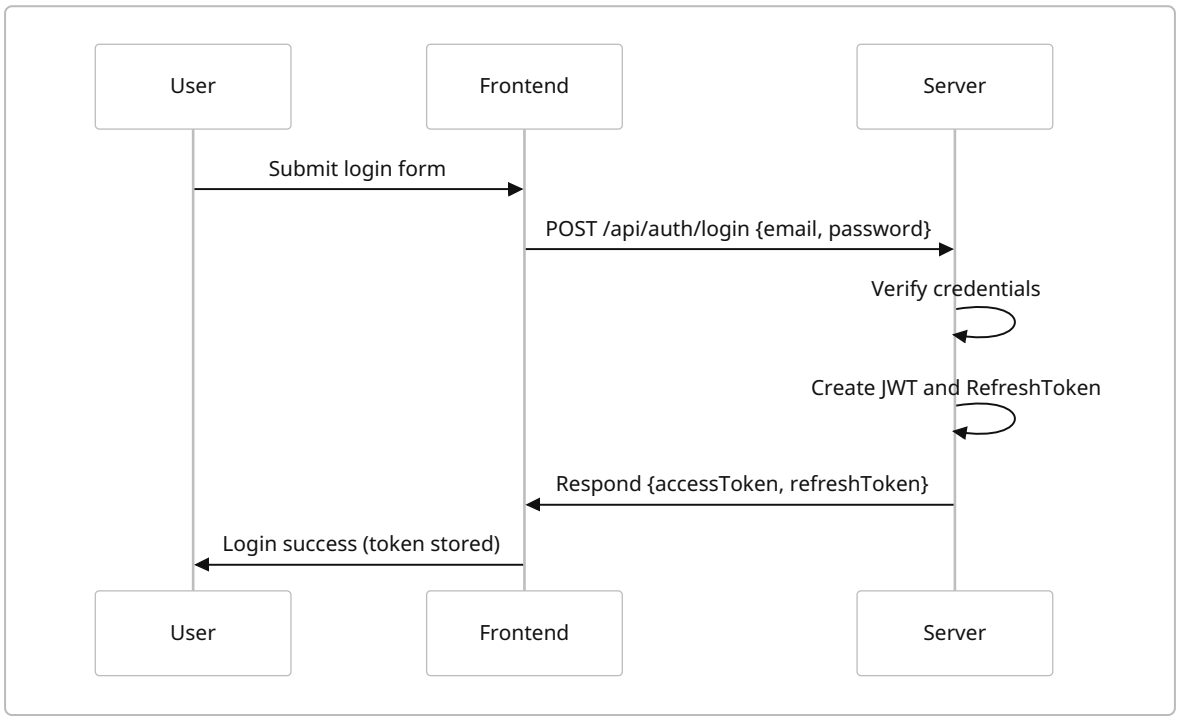
Below are high-level diagrams expressed in **Mermaid** for clarity.

Entity-Relationship Diagram



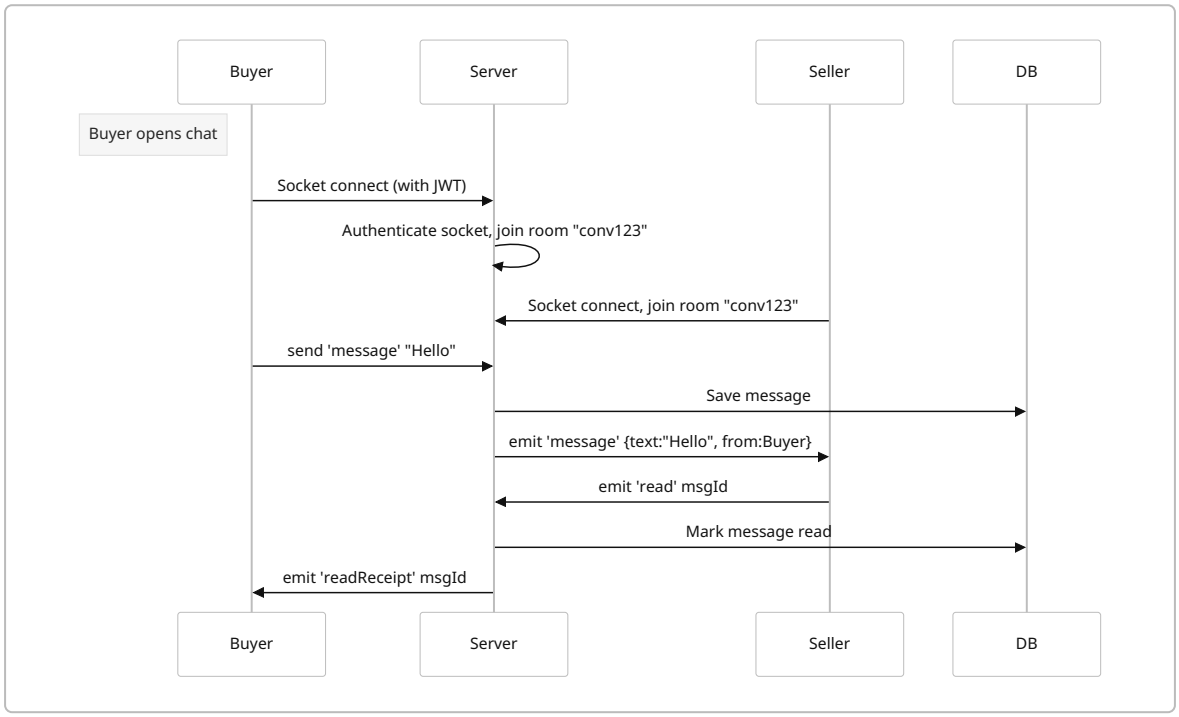
This shows Users linked to Gigs, Orders, Reviews, Conversations, and Wishlists. Each Order ties a buyer, seller, and gig; Reviews and Wishlists link users and gigs; Messages belong to Conversations between buyer and seller.

Authentication Sequence Diagram (JWT login)



After login, the client stores tokens (e.g. in HttpOnly cookies or localStorage).

Real-Time Chat (Socket.IO) Sequence Diagram



This shows two clients (buyer/seller) in a chat “conversation”. Server handles `message` and `read` events, persists data, and forwards events via Socket.IO rooms.

Stripe Payment Sequence Diagram

```
sequenceDiagram
    participant Buyer
    participant Client
    participant Server
    participant Stripe
    Buyer->>Client: Click "Pay"
    Client->>Server: POST /api/stripe/checkout {orderId}
    Server->>Stripe: Create Checkout Session or PaymentIntent
    Stripe-->>Server: session id / client_secret
    Server-->>Client: {sessionId or client_secret}
    Client->>Stripe: Redirect to Stripe or confirm payment
    Stripe->>Server: Webhook (payment_intent.succeeded)
    Server->>DB: Update Order as "paid"
    Note right of Server: Funds held; later payout to seller
```

Funds flow: Buyer pays Stripe, Stripe holds the amount until we call the transfer/payout API to move it to seller. Stripe recommends **destination charges** for marketplaces【5†L18-L24】.

Deployment Architecture Diagram

```
flowchart LR
    subgraph AWS or Cloud (Production)
        direction TB
        NLB[Network LB (HTTPS)] --> ECS[SERVERS (ECS)]
        ECS --> |Routes| "Node/Express app"
        ECS --> |Routes| "React frontend (via NGINX)"
        RDS[(MongoDB Atlas)]
        Redis[(Redis Cache)]
        Cloudinary[(Cloudinary CDN)]
        Stripe[(Stripe API)]
    end
    subgraph Developer_Local
        DevPC[Developer PC]
        DevPC --> DockerCompose["Docker Compose\n(Server + Mongo + Redis +
Client)"]
    end
    ECS -- "Read/Write" --> RDS
    ECS -- "Cache & RateLimit" --> Redis
    ECS -- "Upload/Fetch" --> Cloudinary
    ECS -- "Payment API" --> Stripe
```

This shows production deployment: a Load Balancer routes to containerized Node/React services, connected to managed MongoDB and Redis. Cloudinary and Stripe are external services.

22. Project Plan & Timeline

A rough timeline for development milestones (solo developer vs small team):

Milestone	Tasks	Timeline (Solo)	Timeline (2-3 devs)
Requirements & Design	SRS finalization, ER/sequence diagrams, tech stack review.	Week 1	Week 1
Setup & Auth	Initialize repos, basic UI, JWT + Google OAuth, user model.	Week 2	Week 2 (frontend + backend)
Gig CRUD & DB	Implement Gig model, create/update/delete gigs API.	Week 3	Week 2-3 (parallel)
Search & Pagination	Add text search, filtering, pagination for gigs.	Week 4	Week 3 (merge tasks)
Orders & Stripe	Build Order model; integrate Stripe Connect (checkout & webhooks).	Week 5-6	Week 3-4 (split tasks)
Chat (Socket.IO)	Develop conversation UI and Socket.IO backend.	Week 7	Week 4 (full-stack)
Reviews & Ratings	Add review system post-order, update gig ratings.	Week 8	Week 5 (some overlap)
Seller Dashboard	Analytics queries and dashboard charts.	Week 9	Week 5 (assist)
Wishlist & Notifications	Implement wishlist and email flows (signup, orders, messages).	Week 10	Week 6
Admin Panel	Admin UI and endpoints for moderation (bans, logs).	Week 11	Week 6-7
Theming & Finishing	Dark mode toggle, UI polish, mobile responsiveness.	Week 12	Week 7 (design+front)
Deployment & CI/CD	Dockerize all services; setup GitHub Actions, AWS deploy.	Week 13	Week 8 (ops team?)
Testing & QA	Write tests (unit/integration), bug fixes.	Week 14	Week 9 (testers)
Launch Prep	Final audits, security review, documentation.	Week 15	Week 10

Solo developer might take ~4 months end-to-end, while a *small team* (2-3 members) can parallelize tasks to deploy in 2-3 months. The timeline assumes part-time or moderate pacing; actual time varies by experience and scope changes.

23. Testing Strategy

Automated Tests:

- **Unit Tests:** Use Jest/Mocha for backend logic (e.g. auth, utilities) and React components. Mock external services (Stripe, Cloudinary) using test keys or stubs.
- **Integration Tests:** Use Supertest with an in-memory MongoDB or test database to exercise API endpoints (Auth, Gigs, Orders). Test success and error cases (e.g. unauthorized access returns 401).
- **E2E (End-to-End):** Tools like Cypress or Playwright simulate user flows: registration, gig creation, order placement (using Stripe test card), chat, and review posting. This ensures all layers work together.
- **Sample Test Cases:**
- *Auth:* Register with valid/invalid data; attempt protected routes without token; refresh token flow.
- *Gig:* Create gig as seller, fetch it, update it, forbid another user.
- *Order:* As buyer, create order for a gig; simulate Stripe webhook to mark as paid; verify order status transitions.
- *Chat:* Open two chat sessions (simulated sockets), send message and assert delivery.
- *Admin:* Attempt to ban user and verify role enforcement.

Use continuous integration (GitHub Actions) to run the test suite on every push. Aim for high coverage (>80%). Incorporate linting (ESLint) and type checking (TypeScript or JSDoc) for code quality.

Appendix: Resources and References

- **Official Docs:** Stripe Connect (platforms)【5†L18-L24】, Stripe Security Guide【8†L70-L74】; Cloudinary Node.js SDK【11†L36-L40】【18†L2723-L2728】; Socket.IO Rooms & docs【24†L149-L157】【24†L159-L167】; Redis tutorials (rate limiting)【31†L313-L321】【31†L438-L442】; Redis key eviction policies【33†L146-L154】; GitHub Actions Node.js guide【54†L347-L354】; Google OAuth2 flow【35†L134-L142】【35†L164-L168】; Tailwind dark mode【48†L307-L314】【48†L329-L332】; Express middleware pattern【42†L216-L222】.
- **Sample Code:** Mongoose schema definitions and indexing【39†L1-L8】; API endpoint design (Express router examples) from Express docs; Mermaid diagram syntax (see above code blocks).
- **Environment Variables:**
- `MONGO_URI` – MongoDB connection string
- `JWT_SECRET` – Secret for signing JWTs
- `JWT_EXPIRES_IN` – JWT lifespan (e.g. "15m")
- `REFRESH_TOKEN_SECRET` – Secret for refresh tokens
- `GOOGLE_CLIENT_ID` / `GOOGLE_CLIENT_SECRET` – Google OAuth creds
- `STRIPE_SECRET_KEY` / `STRIPE_PUBLISHABLE_KEY` – Stripe API keys
- `CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_API_KEY`, `CLOUDINARY_API_SECRET` – Cloudinary creds
- `REDIS_URL` – Redis connection URL (with TLS, auth if needed)
- `EMAIL_SMTP_HOST`, `EMAIL_USER`, `EMAIL_PASS` – for email sending
- `NODE_ENV` – "development" or "production"
- (These can be listed in a config table or `.env.example` file.)
- **Priority Matrix (Features):**

Feature	Priority (Must/H/M/L)
JWT Authentication	Must
Payment with Stripe (escrow)	Must
Gig CRUD	Must
Real-time Chat (Socket.IO)	High
Cloudinary Upload	High
Role-Based Access (Buyer/Seller)	Must
Google OAuth Login	Low
Pagination & Search	High
Seller Analytics Dashboard	Medium
Review & Rating System	High
Email Notifications	Medium
Admin Panel	High
Wishlist/Favorites	Low
Dark Mode UI	Low
Docker Deployment	High
Redis Caching (sessions, rate-limit)	Medium
CI/CD (GitHub Actions)	High

(Must = fundamental, High = important, Medium/Low = enhancements.)

This document provides a **comprehensive SRS and implementation blueprint** for the MERN freelance marketplace. It can be expanded into a formal 25–40 page PDF, with figures and code listings, for stakeholders or development teams. The above outline, combined with the cited best practices and code snippets, ensures a robust, scalable solution.
